

DBMS – SELLER AND INVENTORY MANAGEMENT

Project Title: Meesho – Women’s Fashion E-Commerce Database Management System

DBMS TASK WEEK-3

1. Introduction

The Seller and Inventory Management module is designed for the Meesho Women’s Fashion Database Management System. It manages seller information, seller products, stock quantities, and product availability. The module uses relational tables and primary key–foreign key relationships to connect sellers, products, and inventory information.

2. Objectives

- Create Seller and Inventory tables.
- Establish relationships between sellers, products, and stock.
- Maintain seller product information.
- Track available and unavailable products.
- Generate inventory status reports.

3. Existing Product Table

The existing Meesho_Product table stores Product_ID, Product_Name, Category_ID, Price and Stock_Quantity. The Inventory table is linked to this existing product table using Product_ID.

Attribute	Data Type	Key/Constraint	Description
Product_ID			
	INT		
		Primary Key	
			Unique product identifier
Product Name			
	VARCHAR(100)		
		NOT NULL	
			Fashion product name
Category_ID			
	INT		
		Foreign Key	
			References category
Price			
	NUMBER(10,2)		
		NOT NULL	
			Product price
Stock_Quantity			
	INT		
		NOT NULL	
			Available quantity

4. Seller Table Design

Attribute	Data Type	Constraint	Description
Seller_ID			
	NUMBER		
		Primary Key	
			Unique seller identifier
Seller_Name			
	VARCHAR2(100)		
		NOT NULL	
			Seller name
Contact_Number			
	VARCHAR2(15)		
		-	
			Seller contact number
Email			
	VARCHAR2(100)		
		-	
			Seller email

5. Inventory Table Design

Attribute	Data Type	Constraint	Description
Inventory_ID			
	NUMBER		
		Primary Key	
			Unique inventory identifier
Seller_ID			
	NUMBER		
		Foreign Key	
			References Meesho_Seller
Product_ID			
	NUMBER		
		Foreign Key	
			References Meesho_Product
Stock_Quantity			
	NUMBER		
		NOT NULL	
			Current stock quantity
Inventory_Status			
	VARCHAR2(20)		
		-	
			Available/Unavailable

6. Table Creation

```
SQL> CREATE TABLE SELLER3 (  
2     Seller_ID NUMBER PRIMARY KEY,  
3     Seller_Name VARCHAR2(50),  
4     Email VARCHAR2(100) UNIQUE,  
5     Phone_Number VARCHAR2(15) UNIQUE  
6 );
```

Table created.

```
SQL> CREATE TABLE INVENTORY3 (  
2     Inventory_ID NUMBER PRIMARY KEY,  
3     Product_ID NUMBER,  
4     Quantity NUMBER,  
5     Status VARCHAR2(20),  
6     FOREIGN KEY (Product_ID) REFERENCES PRODUCT3(Product_ID)  
7 );
```

Table created.

7. Relationships

- One seller can have many inventory records.
- Each inventory record belongs to one seller.
- Each inventory record is associated with one product.
- Seller_ID references Meesho_Seller.
- Product_ID references Meesho_Product.

SELLER (1) ————— (M) INVENTORY

PRODUCT (1) ————— (M) INVENTORY

8. Data Insertion

```
SQL> INSERT INTO SELLER3 VALUES (101, 'Arun Kumar', 'arun@gmail.com', '9876543210');  
1 row created.  
  
SQL> INSERT INTO SELLER3 VALUES (102, 'Priya Sharma', 'priya@gmail.com', '9876543211');  
1 row created.  
  
SQL> INSERT INTO SELLER3 VALUES (103, 'Rahul Raj', 'rahul@gmail.com', '9876543212');  
1 row created.  
  
SQL> INSERT INTO SELLER3 VALUES (104, 'Divya S', 'divya@gmail.com', '9876543213');  
1 row created.  
  
SQL> INSERT INTO SELLER3 VALUES (105, 'Karthik M', 'karthik@gmail.com', '9876543214');  
1 row created.
```

```

SQL> INSERT INTO INVENTORY3 VALUES (301, 201, 10, 'Available');
1 row created.

SQL> INSERT INTO INVENTORY3 VALUES (302, 202, 15, 'Available');
1 row created.

SQL> INSERT INTO INVENTORY3 VALUES (303, 203, 0, 'Unavailable');
1 row created.

SQL> INSERT INTO INVENTORY3 VALUES (304, 204, 20, 'Available');
1 row created.

SQL> INSERT INTO INVENTORY3 VALUES (305, 205, 0, 'Unavailable');
1 row created.

```

9. Maintain Seller Product Information

```

SQL> SELECT
2     s.Seller_ID,
3     s.Seller_Name,
4     p.Product_ID,
5     p.Product_Name,
6     i.Quantity
7 FROM SELLER3 s
8 JOIN PRODUCT3 p
9     ON s.Seller_ID = p.Seller_ID
10 JOIN INVENTORY3 i
11     ON p.Product_ID = i.Product_ID;

```

SELLER_ID	SELLER_NAME		PRODUCT_ID
		QUANTITY	
101	Arun Kumar		201
Laptop		10	
102	Priya Sharma		202
Smartphone		15	
103	Rahul Raj		203
Headphones		0	
104	Divya S		204
Keyboard		20	
105	Karthik M		205
Smart Watch		0	

10. Track Available and Unavailable Products

10.1 Available Products

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     i.Quantity
  5 FROM PRODUCT3 p
  6 JOIN INVENTORY3 i
  7     ON p.Product_ID = i.Product_ID
  8 WHERE i.Quantity > 0;
```

PRODUCT_ID	PRODUCT_NAME	QUANTITY
201	Laptop	10
202	Smartphone	15
204	Keyboard	20

10.2 Unavailable Products

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     i.Quantity
  5 FROM PRODUCT3 p
  6 JOIN INVENTORY3 i
  7     ON p.Product_ID = i.Product_ID
  8 WHERE i.Quantity = 0;
```

PRODUCT_ID	PRODUCT_NAME	QUANTITY
203	Headphones	0
205	Smart Watch	0

10.3 Update Product

```
SQL> UPDATE Meesho_Inventory
  2 SET Stock_Quantity = 10, Inventory_Status = 'Available'
  3 WHERE Product_ID = 101;
1 row updated.
```

```
SQL> COMMIT;
```

Commit complete.

```
SQL> SELECT * FROM Meesho_Inventory;
```

INVENTORY_ID	PRODUCT_ID	STOCK_QUANTITY	INVENTORY_STATUS
1	101	10	Available
2	102	25	Available
3	103	0	Unavailable
4	104	40	Available
5	105	15	Available

```
5 rows selected.
```

11. Inventory Status Reports

11.1 Product wise Inventory Report

```
SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     i.QUANTITY,
  5     i.STATUS
  6 FROM PRODUCT3 p
  7 JOIN INVENTORY3 i
  8 ON p.Product_ID = i.Product_ID
  9 WHERE i.QUANTITY = 0;
```

PRODUCT_ID	PRODUCT_NAME	QUANTITY
203	Headphones	0
205	Smart Watch	0

11.2 Availability Status Report

```
SQL> SELECT
  2     i.Inventory_ID,
  3     s.Seller_Name,
  4     p.Product_Name,
  5     p.Price,
  6     i.Quantity,
  7     CASE
  8         WHEN i.Quantity > 0 THEN 'Available'
  9         ELSE 'Unavailable'
 10     END AS Stock_Status,
 11     i.Status
 12 FROM INVENTORY3 i
 13 JOIN PRODUCT3 p
 14 ON i.Product_ID = p.Product_ID
 15 JOIN SELLER3 s
 16 ON p.Seller_ID = s.Seller_ID
 17 ORDER BY i.Inventory_ID;
```

INVENTORY_ID	SELLER_NAME	PRODUCT_NAME	PRICE	QUANTITY
301	Arun Kumar	Laptop	55000	10
302	Priya Sharma	Smartphone	25000	15
303	Rahul Raj	Headphones	2500	0
304	Divya S	Keyboard	1500	20
305	Karthik M	Smart Watch	4000	0

11.3 Total Stock by Seller

```
SQL> SELECT
2     CASE
3         WHEN QUANTITY > 0 THEN 'Available'
4         ELSE 'Unavailable'
5     END AS Stock_Status,
6     COUNT(*) AS Total_Products
7 FROM INVENTORY3
8 GROUP BY
9     CASE
10        WHEN QUANTITY > 0 THEN 'Available'
11        ELSE 'Unavailable'
12    END;
```

```
STOCK_STATU  TOTAL_PRODUCTS
-----
Available      3
Unavailable    2
```

12. Expected Inventory Status Report

Seller	Product	Stock	Status
Fashion Hub			
	Cotton Saree		
		25	
			Available
Style Collection			
	Printed Kurti		
		30	
			Available
Trendy Store			
	Western Top		
		20	
			Available
Fashion Hub			
	Fashion Earrings		
		40	
			Available
Style Collection			
	Designer Kurti		
		15	
			Available

13. Conclusion

The Seller and Inventory Management module extends the existing Meesho Women's Fashion database by maintaining seller details and inventory information. Primary key and foreign key constraints establish relationships between sellers, products and stock. SQL queries are used to maintain seller product information, track available and unavailable products, and generate inventory status reports.